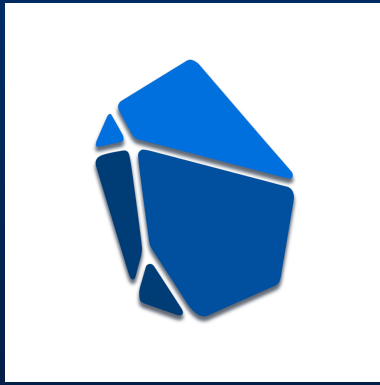




Cobalt Task

Document de vision strategique

D'un assistant vocal personnel
a une plateforme d'agent IA self-hosted
accessible a tous

Fevrier 2026 - Document interne

Sommaire

1	Ou en est Cobalt aujourd'hui	<i>Etat des lieux technique</i>
2	La vision : Cobalt Agent Platform	<i>Ce que le projet peut devenir</i>
3	Le stack IA cible	<i>Pipeline voix de nouvelle generation</i>
4	Architecture serveur self-hosted	<i>Backend multi-utilisateurs</i>
5	Positionnement concurrentiel	<i>Pourquoi Cobalt a sa place</i>
6	Roadmap et couts	<i>Plan de route en 5 phases</i>

Ou en est Cobalt aujourd'hui

Etat des lieux technique et produit

Architecture actuelle

Cobalt Task est aujourd'hui un assistant vocal 100% client-side, tourne sur Android, sans aucun serveur propre. Le telephone fait tout : capturer la voix, transcrire, analyser, executer les actions, et synchroniser avec Google.

Pipeline actuel

1. CAPTURE AUDIO

Bracelet BLE (nRF52840) ou micro du telephone
ADPCM 4-bit, 16 kHz mono -> decompression PCM sur le telephone

V

2. TRANSCRIPTION (cloud)

Groq API - Whisper Large V3
Latence : ~500ms | Cout : ~0.006\$/min | Langues : 99

V

3. ANALYSE IA (cloud)

Groq API - Llama 3.1 8B Instant
Classification en 1 shot (few-shot prompting)
5 categories notes + 10 intents actions | Temperature 0.1-0.3

V

4. EXECUTION (local)

Actions Android natives (SMS, appels, alarmes, volume, GPS...)
+ Sync Google (Tasks, Calendar, Contacts, Docs)
+ Spotify Web API

Forces et limites

Forces

Zero serveur a maintenir

Limites

1 seule cle API partagee (rate limits)

Fonctionne offline (actions locales)

	Aucune memoire conversationnelle
Integration Google profonde	Classification shallow (pas de raisonnement)
Hardware BLE custom (bracelet)	Pas de personnalisation par utilisateur
15+ actions vocales differentes	Sync Google unidirectionnelle
TTS de confirmation ecran verrouille	Cles API dans l'APK (securite)

*En resume : Cobalt est un excellent prototype single-user.
Pour en faire une plateforme multi-utilisateurs, il faut un cerveau central sur un serveur.*

La vision : Cobalt Agent Platform

Un assistant personnel IA, self-hosted, open, intelligent

Le concept

L'idée : transformer Cobalt d'un outil personnel en une plateforme d'agent IA que chacun peut héberger soi-même. Un assistant qui tourne sur TON serveur (ou un VPS à 5 euros/mois), auquel tes amis se connectent via l'app Cobalt sur leur téléphone.

Imagine une fusion entre :

- **OpenClaw** : un agent IA self-hosted qui a explosé (400 000 utilisateurs en 2 mois)
- **Gemini / Bixby** : un assistant quotidien intégré au smartphone
- **MCP (Model Context Protocol)** : un standard d'intégration ouvert

La différence avec les assistants existants :

Cobalt	Gemini / Siri	OpenClaw
Self-hosted (tes données)	Cloud (données chez Google/Apple)	Self-hosted
App smartphone native	App smartphone native	Telegram/Signal (pas d'app)
Actions Android profondes	Actions profondes	Fichiers + web (pas mobile)
Bracelet BLE hardware	Pas de hardware tiers	Pas de hardware
Open + personnalisable	Ferme (black box)	Open source
Français natif	Anglais d'abord	Multi-langue

Le positionnement unique de Cobalt : le seul assistant vocal self-hosted avec une app smartphone native ET du hardware custom (bracelet BLE).

Les 3 piliers de la vision

Pilier 1 : Le Cerveau (serveur IA)

Un backend intelligent qui héberge le LLM, gère la mémoire conversationnelle, les profils utilisateurs, et orchestre les outils via MCP. Chaque utilisateur a son contexte, ses préférences, son historique.

Pilier 2 : Le Corps (app Cobalt + bracelet)

L'application Android reste le point de contact avec le monde reel : micro, haut-parleur, capteurs, actions natives (SMS, appels, GPS, alarmes). Le bracelet ajoute un input hardware mains-libres. Le serveur dit QUOI faire, le telephone fait.

Pilier 3 : Les Mains (integrations MCP)

Via le standard MCP (Model Context Protocol), le serveur peut se connecter a n'importe quel outil : Google Tasks, Notion, Slack, domotique, API meteo, Spotify... sans ecrire de code specifique pour chaque service. MCP est devenu le standard de l'industrie (Linux Foundation, soutenu par OpenAI, Google, Microsoft).

Le stack IA cible

Pipeline voix de nouvelle generation

Pipeline voix evolue

Le pipeline actuel (STT -> classification -> action) est correct mais limité. Le pipeline cible ajoute des couches d'intelligence entre chaque etape.

1. VAD - Voice Activity Detection

Silero VAD (30ms, open-source)
Detecte quand l'utilisateur parle/s'arrete -> decoupe l'audio en segments
Elimine les silences, optimise les appels STT

V

2. STT - Speech-to-Text

Cloud : Deepgram Nova-3 (<300ms, streaming) ou Groq Whisper
Local : Sherpa ONNX (offline fallback)
Nouveau : gpt-4o-transcribe (meilleur WER global)
Streaming = resultats partiels en temps reel

V

3. NLU + CONTEXTE (nouvelle couche)

Enrichissement du texte AVANT le LLM :

- Injection du contexte utilisateur (5 derniers messages, profil, preferences)
- Resolution d'entites ("maman" -> Marie Dupont, 06 12 34 56 78)
- Detection d'emotion (frustration, urgence, question)
- RAG : recherche dans l'historique ("comme la derniere fois")

V

4. LLM - Raisonnement

Modele principal : Claude Sonnet / Llama 3.3 70B / Mixtral (self-hosted)
Chain-of-thought pour les requetes complexes
Multi-turn : "Annule ca" comprend le contexte
Tool calling natif : le LLM decide quels outils appeler via MCP

V

5. ORCHESTRATEUR D'ACTIONS

Route les decisions du LLM vers :

- Actions locales (telephone) : SMS, appels, alarmes, GPS
- Actions cloud (serveur) : Google, Spotify, Notion, domotique
- Actions MCP : n'importe quel serveur MCP connecte

Gere les confirmations, les rollbacks, le retry

V

6. TTS - Text-to-Speech

Kokoro TTS v1.0 (82M params, Apache 2.0, francais natif)

96x temps reel, self-hostable, 54 voix

Alternative : ElevenLabs (meilleure qualite, payant)

Streaming : commence a parler avant la fin de la generation

Les nouvelles couches-cles

Couche NLU + Contexte (entre STT et LLM)

C'est LA couche qui transforme un classifieur basique en assistant intelligent. Aujourd'hui, le LLM recoit le texte brut et doit tout deviner. Demain :

Exemple actuel :

Utilisateur : "Dis-lui que j'arrive"

LLM recoit : "Dis-lui que j'arrive" -> Echec (qui est "lui" ?)

Exemple avec couche NLU :

Utilisateur : "Dis-lui que j'arrive"

NLU injecte : Dernier contact mentionne = Pierre (il y a 2 min)

LLM recoit : [Contexte: dernier contact = Pierre Dupont, 06...] "Dis-lui que j'arrive"

-> SMS a Pierre : "J'arrive"

Memoire conversationnelle

3 niveaux de memoire, comme un humain :

1. Memoire de travail (session) : les 5-10 derniers echanges. Permet le multi-turn.
2. Memoire episodique (jours) : historique des 30 derniers jours. Permet "comme la derniere fois".
3. Memoire semantique (permanente) : profil utilisateur, preferences, contacts frequents. Permet la personnalisation.

RAG (Retrieval-Augmented Generation)

Avant chaque appel au LLM, le serveur cherche dans l'historique de l'utilisateur les informations pertinentes. Si tu dis "ajoute du lait a la liste", le RAG retrouve ta derniere liste de courses et y ajoute le lait au lieu d'en creer une nouvelle.

Architecture serveur self-hosted

Backend multi-utilisateurs avec inference locale

Vue d'ensemble

Le serveur Cobalt est le cerveau central. Il reçoit les requêtes de l'app, raisonne, et renvoie des instructions d'action. L'app reste le corps qui exécute.

COBALT SERVER

- FastAPI (Python) ou Rust (performance)
- Auth : JWT tokens par utilisateur
- Inference : vLLM (Llama 70B Q4 ou Mixtral) sur GPU
- STT : Whisper self-hosted ou Deepgram API
- TTS : Kokoro self-hosted (82M params)
- Mémoire : PostgreSQL + pgvector (RAG)
- Outils : MCP servers (Google, Spotify, Notion...)
- File d'attente : Redis (jobs async)
- Monitoring : Prometheus + Grafana

COBALT APP (Android)

- Client léger :
- Capture audio (micro / bracelet BLE)
- Envoie audio au serveur via WebSocket
- Reçoit les instructions d'action
- Exécute localement (SMS, appels, alarmes, GPS)
- Cache local SQLite (mode hors-ligne)
- TTS local en fallback

Options d'hébergement

Config	GPU	Users max	Cout/mois	Modele
VPS basique	Aucun (CPU)	5-10	5-15 EUR	Llama 3B Q4
VPS GPU	RTX 4090 24GB	10-30	50-250 EUR	Llama 8B Q4
Serveur dédié	RTX 4090 24GB	20-50	250-500 EUR	Llama 70B Q4
Cloud GPU	A100 80GB	50-100	500-2000 EUR	Llama 70B FP16

Hybride

CPU local + API

10-100

100-300 EUR

Cloud fallback

L'option hybride est la plus realiste pour commencer : un petit serveur CPU qui gere l'orchestration et la memoire, avec des appels API (Groq, Deepgram) pour l'inference lourde. Quand le volume justifie un GPU, on migre l'inference en local.

Architecture hybride recommandee (Phase 1)

Pour servir 10-30 utilisateurs sans GPU :

SERVEUR (VPS 4 vCPU, 8 Go RAM, ~15 EUR/mois)
FastAPI : orchestration, auth, memoire, routing
PostgreSQL : utilisateurs, historique, profils
Redis : cache + file d'attente
Appels API : Groq (STT + LLM) / Deepgram (STT streaming)

Cout estime pour 10 utilisateurs actifs :

Poste	Cout/mois	Notes
VPS (Hetzner/OVH)	15 EUR	4 vCPU, 8 Go RAM
Groq API	0-5 EUR	Free tier genereux
Deepgram STT	0-10 EUR	Free tier 200h/mois
PostgreSQL	0 EUR	Sur le meme VPS
Domaine + SSL	1 EUR	Let's Encrypt gratuit
TOTAL	15-30 EUR/mois	

15 a 30 EUR par mois pour servir 10 amis avec un assistant IA personnel complet.
C'est moins cher qu'un abonnement Netflix.

Securite et multi-tenancy

Chaque utilisateur a :

- Un compte avec JWT (login email + mot de passe, ou Google SSO)
- Ses propres tokens Google/Spotify (stockes chiffres cote serveur)
- Son propre espace memoire (historique, preferences, contacts)
- Son propre quota d'appels API (rate limiting par utilisateur)

Les cles API (Groq, Deepgram, Maps) restent sur le serveur, jamais dans l'APK. L'app ne connait que l'URL du serveur et le token JWT de l'utilisateur.

Point d'attention RGPD : le serveur stocke des transcriptions vocales et des donnees personnelles (contacts, calendrier). Prevoir une politique de confidentialite et un mecanisme de suppression des donnees.

Positionnement concurrentiel

Pourquoi Cobalt a sa place sur le marche

Paysage concurrentiel 2026

Les geants : Gemini, Siri, Bixby

Google Gemini est devenu l'assistant par défaut sur Samsung Galaxy S26. Siri est en retard : la version IA est repoussée au printemps 2026 (Apple a signé un accord d'1 Md\$/an avec Google pour utiliser Gemini). Bixby est en retrait stratégique.

Leur faiblesse commune : ils sont fermes, opaques, et centrent tout sur leur écosystème. Impossible de savoir ce qu'ils font de tes données. Impossible de les personnaliser.

Le phénomène OpenClaw

OpenClaw (novembre 2025) a prouvé qu'il existe une demande massive pour un assistant IA self-hosted. 400 000 utilisateurs en 2 mois, 60 000 étoiles GitHub en 72h. Mais il a des limites : pas d'app native, pas d'intégration téléphone, 512 vulnérabilités de sécurité trouvées en audit.

Les startups voix

ElevenLabs (3.3 Md\$ de valorisation), Vapi, Pipecat... se concentrent sur les agents téléphoniques pour entreprises (call centers, support client). Aucun ne cible l'assistant personnel sur smartphone.

Le créneau de Cobalt

Cobalt occupe un créneau unique : le SEUL projet qui combine :

- Self-hosted (données chez toi)
- App smartphone native (pas un chatbot Telegram)
- Actions Android profondes (SMS, appels, alarmes, GPS)
- Hardware custom (bracelet BLE)
- Français natif comme langue première
- Open source et personnalisable

Ce créneau est vide. Ni Google, ni Apple, ni OpenClaw ne l'occupent.

5 niches a forte demande

1. Les utilisateurs soucieux de leur vie privée (post-Snowden, anti-GAFAM)
2. Le marché francophone mal servi par les assistants anglophones
3. Les bricoleurs / makers qui veulent personnaliser leur assistant
4. Les petites équipes / familles qui veulent un assistant partagé
5. Les utilisateurs de montres connectées / wearables DIY

Roadmap et couts

Plan de route en 5 phases

Vue d'ensemble des phases

Phase	Objectif	Duree	Cout infra
1	Backend API Gateway	2-3 semaines	15 EUR/mois (VPS)
2	Memoire + contexte	1-2 semaines	+0 EUR
3	Profils + personnalisation	2 semaines	+0 EUR
4	Integrations MCP	2-3 semaines	+0-10 EUR
5	Inference self-hosted	3-4 semaines	+50-250 EUR (GPU)

Phase 1 : Backend API Gateway (fondation)

Objectif : extraire l'intelligence du telephone vers un serveur central. L'app devient un client léger.

Livrables :

- Serveur FastAPI avec auth JWT
- Endpoints : /transcribe, /analyze, /execute
- Proxy des appels Groq (les clés API quittent l'APK)
- Base PostgreSQL : utilisateurs, sessions
- L'app appelle le serveur au lieu de Groq directement

Impact : les clés API ne sont plus dans l'APK. Chaque utilisateur a son propre compte. Le rate limiting est géré par le serveur, plus par Groq.

Phase 2 : Memoire conversationnelle

Objectif : le LLM se souvient de ce que l'utilisateur a dit avant.

Livrables :

- Table "conversation_history" (user_id, text, intent, result, timestamp)
- Injection des 5 derniers messages dans le prompt LLM
- "Annule ça" fonctionne enfin (le LLM voit le contexte)
- "Comme la dernière fois" retrouve l'historique pertinent

Avant : "Annule ça" -> rien (pas de contexte)
Après : "Annule ça" -> annule le rendez-vous cree il y a 30 secondes

Phase 3 : Profils et personnalisation

- Objectif : le LLM connait l'utilisateur et s'adapte.
- Livrables :
- Profil utilisateur : contacts frequents, apps preferees, horaires habituels
 - Alias de contacts : "maman" = Marie Dupont, 06...
 - Apprentissage des preferences : "toujours envoyer par WhatsApp a Pierre"
 - A/B testing de prompts (mesurer le taux de correction)

Phase 4 : Integrations MCP

- Objectif : connecter le serveur a n'importe quel service via le standard MCP.
- Livrables :
- Client MCP dans le serveur (consomme des MCP servers)
 - MCP servers pour : Google Tasks, Calendar, Contacts, Spotify
 - Possibilite d'ajouter Notion, Slack, Home Assistant, Todoist...
 - Le LLM choisit automatiquement quel outil appeler (tool calling)

MCP est le standard de l'industrie (Linux Foundation). En exposant Google Tasks comme MCP server, tu rends Cobalt compatible avec tout l'ecosysteme MCP : Claude Desktop, Cursor, tout agent compatible.

Phase 5 : Inference self-hosted

- Objectif : remplacer les appels API payants par de l'inference locale sur GPU.
- Livrables :
- vLLM pour servir Llama 3.3 70B Q4 (ou Mixtral) sur RTX 4090
 - Whisper self-hosted pour la transcription
 - Kokoro TTS pour la synthese vocale (open source, francais)
 - Zero dependance a des API tierces (full self-hosted)

Budget hardware :

Composant	Prix	Ce que ca fait
RTX 4090 24 Go	~1 600 EUR	LLM 70B Q4 + Whisper + TTS
Ou : cloud GPU	~250 EUR/mois	Meme chose sans achat

Alternative CPU

0 EUR (VPS)

Llama 3B Q4 (basique)

La Phase 5 est optionnelle. L'architecture hybride (Phase 1-4) avec des appels API (Groq free tier) fonctionne tres bien pour 10-30 utilisateurs a ~15-30 EUR/mois.

Conclusion

Cobalt Task est déjà un assistant vocal fonctionnel et unique. La vision décrite ici transforme ce prototype en une plateforme d'agent IA complète :

Aujourd'hui : Un téléphone qui écoute et agit (single-user, clés dans l'APK)

Phase 1-2 : Un serveur intelligent qui écoute, comprend le contexte, et agit (multi-user)

Phase 3-4 : Un agent personnel qui te connaît et se connecte à tout (MCP)

Phase 5 : Une plateforme 100% self-hosted, zero dependance (full souverainete)

Le marché de l'assistant personnel IA est dominé par des écosystèmes fermes.
Cobalt propose une alternative ouverte, self-hosted, avec du hardware custom.
C'est un créneau vide, avec une demande prouvée (OpenClaw : 400K utilisateurs).

Coût d'entrée : 15 EUR/mois pour 10 utilisateurs.
Coût complet : 250 EUR/mois pour 50 utilisateurs avec inférence locale.

La fondation est là. Le reste, c'est de l'exécution.